

Simulazione del comportamento di stormi

Relazione progetto 2024/2025

Filippo Righi e Tommaso Zauli

30 Giugno 2025

1 Introduzione

Il progetto si basa sul tema proposto riguardante la simulazione del comportamento di stormi, o boids, in base a regole di separazione, coesione e allineamento. L'obiettivo è osservare il comportamento di uno stormo date le interazioni locali tra i membri

2 Scelte progettuali e implementative

Il progetto è scritto in C++, è sviluppato su più file (boids.cpp, boids.hpp, boids.test.cpp, visualizzazione.cpp, visualizzazione.hpp, main.cpp, main.grafica.cpp) responsabili della simulazione e della grafica del progetto. Di seguito vengono elencate *struct* e *class* utilizzate:

- *vettore* (*struct* in boids.cpp e boids.hpp): struttura fondamentale che rappresenta un vettore bidimensionale. Include operazioni essenziali come somma, sottrazione, prodotto per scalare, calcolo del modulo e normalizzazione. Serve per la gestione di posizioni e velocità all'interno della simulazione.
- *Boid* (*classe* in boids.cpp e boids.hpp): rappresenta il singolo agente, il boid, all'interno di uno stormo. Ogni boid possiede una posizione e una velocità.
- *Allboids* (*classe* in boids.cpp e boids.hpp): Questa classe gestisce l'intera popolazione di boid e implementa le tre regole fondamentali di Reynolds: **Separazione**: evita gli affollamenti spingendo i boids a mantenere una distanza minima dagli altri. **Allineamento**: spinge i boids a dirigersi nella stessa direzione dei vicini, favorendo l'uniformità del movimento. **Coesione**: fa sì che i boid si muovano verso il centro di massa dei vicini, promuovendo la formazione di uno stormo compatto. Il metodo `aggiornaBoids()` orchestra l'applicazione di queste regole per tutti i boids nella simulazione.

- *Visualizzazione* (classe in `visualizzazione.cpp` e `visualizzazione.hpp`): Questa classe è responsabile del rendering grafico della simulazione. Utilizza la libreria SFML per disegnare ogni boid come un cerchio colorato su una finestra grafica, traducendo le posizioni logiche in rappresentazioni visive.

Il progetto è composto da due eseguibili *main*:

- *main.cpp* è dedicato alla simulazione senza grafica. Inizializza i boid e li aggiorna per un numero definito di passi temporali, calcolando e stampando metriche statistiche come la distanza media e la velocità media dello stormo, con relative deviazioni standard, ad ogni passo.
- *main.grafica.cpp* che gestisce la simulazione con interfaccia grafica. Inizializza l'ambiente SFML, popola lo stormo e, in un ciclo continuo, aggiorna lo stato dei boids e ne visualizza la posizione sulla finestra, offrendo una rappresentazione dinamica del fenomeno.

3 Librerie esterne

Il progetto utilizza la libreria SFML per la gestione della grafica e della finestra. Per installare SFML su Ubuntu è possibile digitare `sudo apt install libsFML-dev` nel terminale.

4 Istruzioni per l'esecuzione

4.1 Esecuzione della simulazione

Eseguendo il comando `./boids.sim` avrà inizio la simulazione del comportamento di stormi, senza interfaccia grafica.

4.2 Esecuzione della parte grafica

Eseguendo il comando `./boids.sfm` si aprirà una finestra che mostrerà la simulazione visiva dei boids, secondo parametri specifici richiesti in input.

5 Parametri di input e formato output

5.1 I/O simulazione

Vengono richiesti in input i parametri N , d , ds , s , a , c , dt , che rappresentano rispettivamente il numero di boids nella simulazione, il raggio visuale per riconoscere i boids vicini, il raggio visuale che attiva la separazione, il peso del coefficiente di separazione, il peso del coefficiente di allineamento, il peso del coefficiente di coesione e l'intervallo di tempo in cui valutare il comportamento dello stormo. Esempi di input:

- 100 100 50 10 0.01 0.01 1
- 100 50 20 1 0.1 0.1 1
- 50 20 5 1 1 1 1
- 500 100 50 1 1 1 1

Per ogni passo temporale (100 aggiornamenti) il programma stampa in output una riga con: in ordine l'intervallo di tempo, la posizione media dello stormo e relativa deviazione standard, la velocità media dello stormo e relativa deviazione standard. Esempio di output:

- 0 103.376 49.6425 1.67621 0.576367
- 1 103.049 49.6918 1.63814 0.599508
- ...

5.2 I/O parte grafica

Vengono richiesti in input i parametri N , d , ds , s , a , c e dt , come sopra, prima dell'avvio della finestra grafica. Esempio di input:

- $N=100$ $d=100$ $ds=50$ $s=10$ $a=0.01$ $c=0.01$ $dt=1$

Come output restituisce una finestra grafica che mostra una simulazione di interazione tra boids con parametri inseriti in input.

6 Interpretazione dei risultati

6.1 Simulazione boids.sim

Analizzando l'output statistico dell'eseguibile si osserva che tipicamente i valori di posizioni e velocità con rispettive deviazioni standard tendono a stabilizzarsi, simulando la convergenza dei boids in uno stormo coeso. Inoltre si può verificare l'impatto dei parametri s , a e c sulla simulazione: un s elevato porterà a stormi più dispersi, un a elevato tende a far muovere i boids in formazioni più ordinate mentre un c elevato fa sì che si raggruppino maggiormente.

6.2 Grafica boids.sfml

La visualizzazione grafica offre un'interpretazione qualitativa immediata e dinamica. Si osserva la formazione spontanea di stormi con boids che interagiscono tra loro compiendo manovre collettive. I bordi vengono gestiti considerando lo spazio bidimensionale toroidale, dunque un boid che raggiunge il bordo viene teletrasportato dal lato opposto, permettendo alla simulazione di risultare continua. La possibilità di inserire i parametri in input permette di sperimentare i vari comportamenti dei boids al variare dei valori dei parametri.

7 Strategia di test

Per verificare il corretto funzionamento della simulazione è stato realizzato il file `boids.test.cpp` tramite `doctest.h`. Il file è composto da tre *test case* divisi a loro volta in diversi *subcase* con l'obiettivo di controllare la struct vettore e le classi `Boid` e `Allboids` con rispettivi metodi. Per quanto riguarda la grafica è stato eseguito un test visivo che ha confermato la coerenza del funzionamento della simulazione con `sfml`.

8 Informazioni utili

- `maxVel`: è stato impostato un valore di velocità massima dei boids ed è stata verificata la sua coerenza con la classe `Allboids` e con la simulazione.
- metodi `Boid::aggiorna` e `Allboids::aggiornaBoids`: nel progetto sono state implementate due diverse funzioni per l'aggiornamento del sistema. Nello specifico `Boid::aggiorna` aggiorna individualmente ogni boid in base a posizione e velocità e viene usato nella parte grafica del programma per visualizzare lo stato dei singoli boid in tempo reale, includendo la gestione ai bordi. `Allboids::aggiornaBoids` invece calcola contemporaneamente tutte le nuove velocità dei boid, utilizzato per la simulazione da terminale. La prima nella parte grafica consente di disegnare i boid in maniera indipendente rendendo più fluida la simulazione visiva, la seconda nella simulazione testuale garantisce un comportamento sincrono e coerente tra i boids utile per l'interpretazione dei risultati.